

Ex. No. : 05

Date:

Register No : 231701025

Name: Kamalesh R

A python program to implement Multi Layer Perceptron

Aim:

To develop a Python program to implement a **Multi Layer Perceptron with back Propagation** for performing binary classification.

Requirements:

Jupyter Notebook with Python.

PROGRAM:

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

y = np.array([[0],
              [1],
              [1],
              [0]])

np.random.seed(42)
```

```

input_neurons = 2

hidden_neurons = 4

output_neurons = 1

W1 = np.random.uniform(size=(input_neurons, hidden_neurons))

b1 = np.random.uniform(size=(1, hidden_neurons))

W2 = np.random.uniform(size=(hidden_neurons, output_neurons))

b2 = np.random.uniform(size=(1, output_neurons))

lr = 0.1

epochs = 10000

for epoch in range(epochs):

    hidden_input = np.dot(X, W1) + b1

    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, W2) + b2

    final_output = sigmoid(final_input)

    error = y - final_output

    d_output = error * sigmoid_derivative(final_output)

    error_hidden = d_output.dot(W2.T)

    d_hidden = error_hidden * sigmoid_derivative(hidden_output)

    W2 += hidden_output.T.dot(d_output) * lr

    b2 += np.sum(d_output, axis=0, keepdims=True) * lr

    W1 += X.T.dot(d_hidden) * lr

    b1 += np.sum(d_hidden, axis=0, keepdims=True) * lr

print("Final Output after training:")

print(final_output)

```

Output:

Final Output after training:

```
[[0.04690963]  
 [0.95663392]  
 [0.92548675]  
 [0.07177571]]
```

Result:

Thus, the **Multi Layer Perceptron** was successfully implemented in Python and used to perform binary classification.